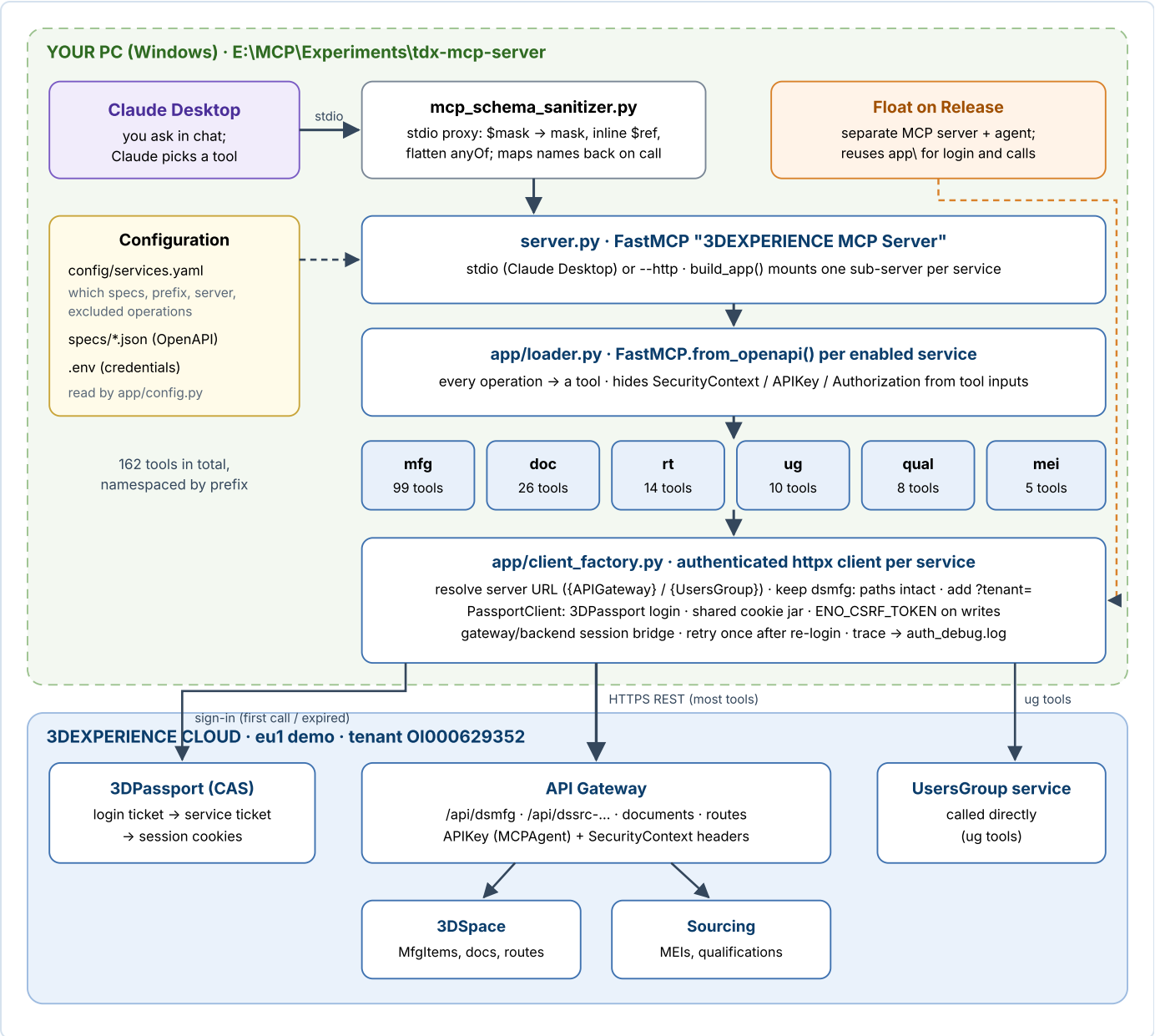
3DX MCP Server

Architecture and request flow of `tdx-mcp-server`, the 3DEXPERIENCE MCP server for Claude Desktop · tenant OI000629352 · October 15, 2026

Purpose

The server exposes 3DEXPERIENCE REST web services to Claude as MCP tools, so you can search, read and change Manufacturing Items, MEIs, qualifications, documents, routes and user groups by asking in chat. Tools are generated **automatically from the Dassault OpenAPI spec files**: adding a web service means dropping in a spec file and adding one entry to `config/services.yaml`, with no new code. All calls go through the **3DEXPERIENCE API Gateway**, signed in through **3DPassport** with credentials kept in `.env`, never passed through the chat.

Architecture diagram



Solid arrows: request path. Dashed: configuration read at start-up, and Float on Release reusing the same client code.

Components

File	Role
`server.py`	Entry point. `python server.py` serves over stdio for Claude Desktop; `--http` serves streamable HTTP. Calls `build_app()`.
`app/loader.py`	For each enabled entry in `services.yaml`: loads the spec, builds its client, and turns every operation into a tool with `FastMCP.from_openapi()`. Applies the exclusions, hides auth headers from tool inputs, and mounts the sub-server under its prefix (`mfg_…`, `mei_…`).
`app/client_factory.py`	Builds the authenticated HTTP client: resolves `{APIGateway}` / `{UsersGroup}` server URLs, keeps `dsmfg:` paths intact, adds `?tenant=`, and handles 3DPassport login, shared session cookies, CSRF tokens for writes, gateway-to-backend session bridging, one retry after re-login and clear error messages. Writes an auth trace to `auth_debug.log`.
`app/config.py`	Reads `.env` (credentials and URLs) and `config/services.yaml`.
`app/mcp_schema_sanitizer.py`	Optional stdio proxy between Claude Desktop and the server. Rewrites tool schemas so the Claude API accepts them (for example `$mask` → `mask`, `dsmfg:MfgItem` → `dsmfg_MfgItem`, inlined `$ref`s, flattened `anyOf`), maps names back on each call, and can dump before/after schemas to `schema_dump/`.
`config/services.yaml`	The registry of web services: spec file, enabled flag, tool prefix, which server to call, and optional excluded tags / operations / paths.
`specs/*.json`	Dassault OpenAPI specs, unchanged.
`.env`	Credentials and URLs. Never committed, and never visible to Claude.

Web services exposed

Prefix	Service	Spec	Called through	Tools
`mfg`	Manufacturing Items and instances (dsmfg)	Manufacturing.json	API Gateway → 3DSpace	99
`doc`	Documents and their files	Document.json	API Gateway → 3DSpace	26
`rt`	Routes and route templates (dsrt)	Route.json	API Gateway → 3DSpace	14
`ug`	User groups and members	Usergroup.json	UsersGroup service, directly	10
`qual`	Qualifications (dssrc)	Qualification.json	API Gateway → Sourcing	8
`mei`	Manufacturer Equivalent Items (dssrc)	MEI.json	API Gateway → Sourcing	5
Total				**162**

`specs/Engineering.json` and `specs/CSRFToken.json` are in the folder but not listed in `services.yaml`, so they expose no tools. Adding an entry for Engineering.json would add Engineering Item tools.

Flow 1 · Start-up

1. Claude Desktop starts the server from `claude_desktop_config.json`, through the schema sanitizer when it's configured that way.

2. `config.py` loads `.env` and `services.yaml`.
3. For each enabled service, `loader.py` loads its spec and `client_factory.py` picks the base URL: the gateway (`…/api/dsmfg/v1`) or the tenant's UsersGroup service.
4. `FastMCP.from_openapi()` turns each operation into a tool, minus any exclusions, and the auth header parameters are removed from the tool inputs.
5. Each sub-server is mounted under its prefix. Claude Desktop lists the 162 tools; the sanitizer rewrites their schemas so the Claude API accepts them.

Flow 2 · A tool call

Example: "Show me the BOM of AJ MA 300", which calls `mfg_GETAll_dsmfgMfgItem_ID_dsmfgMfgItemInstance`.

1. Claude chooses the tool and its arguments (`ID`, `mask`). The sanitizer renames them back to the spec's names (`mask` → `$mask`) and forwards the call.
2. FastMCP maps the tool to its OpenAPI operation and builds the HTTP request: path `/dsmfg:MfgItem/{ID}/dsmfg:MfgItemInstance` plus the query.
3. The client adds the default headers from `.env` (`SecurityContext`, `APIKey`) and `?tenant=OI000629352`.
4. **First call or expired session:** the client signs in to 3DPassport (login ticket, then a service ticket for 3DSpace) and checks the session. Cookies are kept in a shared jar, so later calls and other services reuse them.
5. **Write calls** (POST / PATCH / PUT / DELETE): the client first fetches `ENO_CSRF_TOKEN` from the same backend session and attaches it.
6. The request goes to the API Gateway. If the gateway sends it to a login page (the gateway keeps its own session, and qualifications and MEIs live on the Sourcing service), the client completes that sign-in, bridges the session and retries once.
7. The JSON response goes back through FastMCP and the sanitizer to Claude as the tool result. Errors come back with a plain explanation (for example an expired API key or a wrong SecurityContext), and the details go to `auth_debug.log`.

Configuration (`.env` keys, values not shown)

Key	Purpose
`TDX_USERNAME`, `TDX_PASSWORD`	3DPassport sign-in used by the server
`SecurityContext`	Role.Organization.Collab space sent on every call
`APIKey`	API Gateway key. It's the MCPAgent agent key, so if that key is regenerated, `.env` must be updated.
`TDX_APIGATEWAY_URL`, `TDX_BASE_URL`, `TDX_SPACE_URL`	Gateway and 3DSpace addresses; the 3DSpace one is derived if not set
`TDX_TENANT`, `TDX_PASSPORT_URL`	Tenant ID and 3DPassport address; both derived if not set
`TDX_SERVER_PREFERENCE`, `TDX_VERIFY_SSL`, `TDX_TIMEOUT_SECONDS`	Default server, SSL checking (keep on), request timeout

Adding or removing a web service

1. Put the Dassault OpenAPI spec in `specs/`.
2. Add an entry to `config/services.yaml` with `name`, `file`, `prefix`, `server_preference: APIGateway` and any exclusions.
3. Restart Claude Desktop. To remove a service, set `enabled: false`. To hide individual risky operations, such as deletes, use `exclude_operation_ids`.

What builds on it

- **Float on Release** (`E:\MCP\Experiments\float-on-release`) imports `app\` for its 3DEXPERIENCE calls. It shares the same `.env`, but uses its own Owner security context for replacing instances.
- Any future headless agent, such as the Sourcing agent for revision events, can reuse the same client in the same way.

Notes and limits

- **Tool count:** 162 tools is a lot for Claude to choose from. Excluding operations you never use, especially in `mfg` (99), makes tool choice faster and more accurate.
- **Single identity:** every call runs as the one `.env` user and SecurityContext. Per-call roles would need `SecurityContext` exposed as a tool input.
- **SSO tenants:** username/password sign-in doesn't work if the tenant enforces SSO.
- **Over HTTP:** if the server is exposed with `--http` to other clients, check FastMCP's header-forwarding behaviour first.

Source: files in E:\MCP\Experiments\tdx-mcp-server as of October 15, 2026. Tool counts from schema_dump/tools_sanitized.json.